

Inventory Management System

The **Inventory Management System** is a desktop-based application developed to maintain product information, track stock quantities, search products, and update inventory records efficiently. The system is designed as a simple college-level Python project using a graphical user interface and a local database. It is suitable for academic demonstration as well as basic practical inventory management.

The main objective of this project is to provide a simple and organized way to store product details in a structured database. The system allows users to add new products along with their stock quantities, update stock after purchases or sales, search for specific products, delete obsolete product records, and display available inventory.

The application is developed using **Python 3** as the programming language, **Tkinter** for designing the graphical user interface, and **SQLite3** for database management. The application follows an event-driven desktop application approach. The SQLite database file, named `inventory.db`, is created automatically when the program is executed for the first time.

The system consists of several major modules, including **Product Registration, Product Search, Stock Update, Product Deletion, and Inventory Display**. These modules work together to provide basic inventory management functionality through an easy-to-use graphical interface.

To install and execute the project, Python 3.x must be installed on the computer. The project folder can then be opened in VS Code, IDLE, or any other Python-supported editor. After opening a terminal in the project directory, the application can be started using the command `python inventory_system.py`. Once the program starts, the SQLite database is automatically created, and users can enter sample product records and test the available features.

The main files included in the project are `inventory_system.py`, which contains the main Python source code; `inventory.db`, which is generated automatically to store inventory data; `PROJECT_REPORT.txt`, which contains the detailed academic project report; and `README.txt`, which provides project setup and usage instructions.

The system can be tested by adding multiple product records, entering both normal and edge-case values, selecting records, and testing different operations such as updating and deleting products. The application can also be closed and reopened to verify that the inventory data remains stored in the SQLite database. Screenshots of the main interface and important operations can be included in the project report for academic submission.

As this is an educational project, the system has certain limitations. It is intentionally lightweight and does not provide advanced security features, multi-user authentication, cloud synchronization, or enterprise-level data validation.

In the future, the Inventory Management System can be enhanced by adding features such as **low-stock alerts, supplier management, sales and purchase invoices, barcode integration, dashboards, and graphical charts**. These improvements can make the system more useful for real-world inventory management.

The final project submission should include the Python source file, README file, project report, required screenshots, and a properly tested working application.